Desde cero hasta ChatGPT 🤖

Día 2: Redes recurrentes y tokenización

Ronghao Pan

2 de marzo de 2026

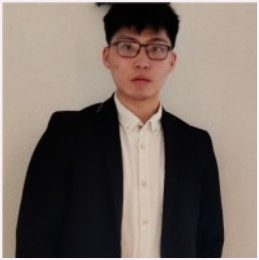
Sobre los docentes 👩‍🏫

José Antonio Hernández López

- Matemático e informático
- Doctor en Informática
- Profesor universitario de la UMU
- Investigación: LLMs para generación de código
- joseantonio.hernandez6@um.es

Ronghao Pan

- Ingeniero informático
- Doctorando e investigador en PLN
- Miembros del TECNOMOD y UMUTeam
- Investigación: LLMs, ASR, VLM
- ronghao.pan@um.es



Objetivo del curso 🎯

Estoy seguro de que muchos de vosotros sabéis

- Crear una red neuronal con, por ejemplo, Keras y entrenarla con un dataset
- Que una MLP son varias capas lineales con ReLU/sigmoid
- Instanciar distintas arquitecturas (MLP, CNN, RNN) según el problema
- Usar ChatGPT
- ...

Objetivo del curso 🎯

Estoy seguro de que muchos de vosotros sabéis

- Crear una red neuronal con, por ejemplo, Keras y entrenarla con un dataset
- Que una MLP son varias capas lineales con ReLU/sigmoid
- Instanciar distintas arquitecturas (MLP, CNN, RNN) según el problema
- Usar ChatGPT
- ...

pero... ¿sabéis...

- cómo funciona una red neuronal por dentro?
- de dónde viene la arquitectura transformer?
- cómo se ha llegado hasta ChatGPT?
- en qué se basan los agentes LLMs?

Objetivo del curso 🎯

Estoy seguro de que muchos de vosotros sabéis

- Crear una red neuronal con, por ejemplo, Keras y entrenarla con un dataset
- Que una MLP son varias capas lineales con ReLU/sigmoid
- Instanciar distintas arquitecturas (MLP, CNN, RNN) según el problema
- Usar ChatGPT
- ...

pero... ¿sabéis...

- cómo funciona una red neuronal por dentro?
- de dónde viene la arquitectura transformer?
- cómo se ha llegado hasta ChatGPT?
- en qué se basan los agentes LLMs?

El **objetivo del curso** es que comprendáis en profundidad el funcionamiento interno de las redes neuronales, yendo un paso más allá de lo que suele encontrarse en tutoriales o recursos generales de Internet. Seguiremos un enfoque progresivo, en el que iremos construyendo poco a poco todo el conocimiento necesario sobre redes neuronales hasta llegar a entender cómo funcionan modelos como ChatGPT.

Roadmap general 🗺️

1. Redes neuronales y Pytorch - **Día 1**
2. Redes recurrentes y tokenización - **Día 2**
3. Mecanismo de atención, Transformers, Pre-training y Fine-tuning - **Día 3**
4. Modelos decoder-only. De GPT a ChatGPT - **Día 4.1**
5. Agentes LLM - **Día 4.2**

Roadmap del día 2 🗺️

1. ¿Qué son los tokenizadores y capas de embeddings?
   - Introducción y Motivación
   - Tokenizadores
   - Capa de embeddings
2. Las redes recurrentes para el procesamiento del lenguaje natural
   - ¿Qué son las redes neuronales recurrentes?
3. Modelos del lenguaje autorregresivos
   - ¿Qué es un modelo de lenguaje?

¿Qué son los tokenizadores y capas de embeddings?

Introducción y Motivación

Introducción y motivación

Introducción

Los modelos de aprendizaje profundo (*deep learning*) que trabajan con lenguaje natural no pueden operar directamente sobre texto. Un texto es una secuencia de símbolos discretos, mientras que las redes neuronales operan sobre **vectores numéricos continuos**.

Sea un texto definido como:

$$x = (c_1, c_2, \ldots, c_N),$$

donde cada c_i es un carácter.
El objetivo es transformar este texto en una representación adecuada para una red o modelo neuronal. Este proceso se divide en dos etapas fundamentales:

1. **Tokenización**: conversión del texto en unidades discretas manejables.
2. **Embeddings**: proyección de esas unidades discretas a un espacio vectorial continuo.

Introducción y motivación

Formalmente

$$x \xrightarrow{\text{tokenizador}} (t_1, \ldots, t_T) \xrightarrow{\text{embeddings}} (e_1, \ldots, e_T), \quad e_i \in \mathbb{R}^d.$$

donde:

- $x \in \mathcal{X}$ es un texto de entrada, definido como una secuencia de caracteres

$$x = (c_1, c_2, \ldots, c_N),$$

- $\mathcal{T} : \mathcal{X} \to \mathcal{V}^T$ es el *tokenizador*, una función determinista que transforma el texto en una secuencia de tokens pertenecientes a un vocabulario finito $\mathcal{V}$.

Introducción y motivación

Formalmente

$$x \xrightarrow{\text{tokenizador}} (t_1, \ldots, t_T) \xrightarrow{\text{embeddings}} (e_1, \ldots, e_T), \quad e_i \in \mathbb{R}^d.$$

donde:

- $(t_1, \ldots, t_T)$ es la secuencia de tokens obtenida tras la tokenización, con
$$t_i \in \mathcal{V}, \quad i = 1, \ldots, T.$$
- $T \in \mathbb{N}$ es la longitud de la secuencia de tokens, que depende tanto del texto de entrada como del esquema de tokenización utilizado.
- $\mathcal{E} : \mathcal{V} \to \mathbb{R}^d$ es la capa de embeddings, que asocia a cada token un vector denso de dimensión d.
- $(e_1, \ldots, e_T)$ es la secuencia de embeddings correspondiente, donde
$$e_i = \mathcal{E}(t_i) \in \mathbb{R}^d, \quad i = 1, \ldots, T.$$
- $d \in \mathbb{N}$ es la dimensión del espacio de embeddings, un hiperparámetro del modelo.

Tokenizadores

Tokenizador

¿Qué es un tokenizador?

Un tokenizador es un función determinista:

$$\mathcal{T} : \mathcal{X} \to \mathcal{V}^T,$$

donde:

- $\mathcal{X}$ es el conjunto de textos posibles,
- $\mathcal{V}$ es un vocabulario finito de tokens,
- T es la longitud de la secuencia de tokens.

En la práctica, el tokenizador devuelve una secuencia de índices enteros:

$$\mathcal{T}(x) = (i_1, i_2, \ldots, i_T), \quad i_k \in \{1, \ldots, |\mathcal{V}|\}.$$

Tokens y vocabulario

Definición

El vocabulario se define como:

$$\mathcal{V} = \{v_1, v_2, \ldots, v_{|\mathcal{V}|}\}.$$

Cada token v_j tiene asignado un identificador entero j. Los tokens pueden representar palabras completas, subpalabras, caracteres, bytes o símbolos especiales.

Por ejemplo, sea el texto de entrada:

$$x = \texttt{"Me gusta la pizza"}.$$

Tras aplicar el tokenizador, se obtiene la secuencia de tokens:

$$(t_1, t_2, t_3, t_4) = (\texttt{"Me"},\ \texttt{" gusta"},\ \texttt{" la"},\ \texttt{" pizza"}).$$

Cada token se asocia a un identificador entero del vocabulario:

$$(i_1, i_2, i_3, i_4) = (201,\ 4501,\ 33,\ 9902).$$

Tokenización por subpalabras

Limitación de tokenizador de palabras completas

Si cada palabra completa fuera un token, el tamaño del vocabulario crecería sin límite:

$$|\mathcal{V}| \to \infty,$$

lo que provocaría un número excesivo de parámetros y problemas de palabras fuera de vocabulario (*Out-Of-Vocabulary*, OOV).

La tokenización por subpalabras permite descomponer una palabra compleja, por ejemplo:

$$\text{“electroencefalografista”} \to (t_1, t_2, \ldots, t_k),$$

garantizando que cualquier texto pueda representarse como combinación de tokens conocidos.

Tokenización por subpalabras

Definición

- La *tokenización por subpalabras* es una estrategia intermedia entre la tokenización a nivel de palabras y la tokenización a nivel de caracteres.
- Su objetivo es construir un **vocabulario finito** que permita representar cualquier texto como una secuencia de unidades reutilizables, capturando regularidades morfológicas y reduciendo el problema de palabras fuera de vocabulario.

Formalmente, sea un texto de entrada:

$$x = (c_1, c_2, \ldots, c_N),$$

donde cada c_i es un carácter. La tokenización por subpalabras busca una segmentación:

$$x \longrightarrow (t_1, t_2, \ldots, t_T),$$

tal que:

$$t_i \in \mathcal{V}, \quad i = 1, \ldots, T,$$

y el vocabulario $\mathcal{V}$ es de tamaño finito y predefinido.

Tokenización por subpalabras

Segmentación óptima

La *tokenización por subpalabras* puede formularse como un problema de optimización. Dado un vocabulario $\mathcal{V}$, se busca la segmentación que maximiza una función objetivo:

$$(t_1, \ldots, t_T) = \arg \max_{(u_1,\ldots,u_K)} P(u_1, \ldots, u_K),$$

sujeta a:

$$u_j \in \mathcal{V}.$$

Dependiendo del algoritmo de tokenización, se definen diferente funciones objetivos:

- **BPE**: maximiza frecuencia de fusiones de pares.
- **Unigram LM**: maximiza la probabilidad del texto.
- **Byte-level BPE**: garantiza cobertura total del texto Unicode.

Algoritmos de tokenización por subpalabras

- Existen diversos algoritmos para construir tokenizadores de subpalabras.
- Todos comparten el objetivo de **aprender un vocabulario finito** que permita segmentar cualquier texto de forma **eficiente y generalizable**.
- Los algoritmos más utilizados en modelos de lenguaje son:
  - **Byte Pair Encoding (BPE)**
  - **WordPiece**
  - **Tokenización a nivel de bytes (Byte-level BPE)**
  - **Unigram Language Model**

Byte Pair Encoding (BPE)

Definición

El algoritmo *Byte Pair Encoding* (BPE) es un **método iterativo** basado en la frecuencia de **coocurrencia de símbolos adyacentes**. Originalmente propuesto como un algoritmo de compresión, fue adaptado posteriormente al procesamiento del lenguaje natural.

Inicialización: Se parte de una representación inicial del texto como una secuencia de símbolos básicos, típicamente caracteres o bytes. El vocabulario inicial se define como:

$$\mathcal{V}_0 = \{c_1, c_2, \ldots, c_M\},$$

donde cada c_i es un carácter del alfabeto.

Byte Pair Encoding (BPE)

Paso iterativo: En cada iteración, se identifica el par de tokens adyacentes más frecuente:

$$(a, b) = \arg\max_{(x,y)} \text{freq}(x, y),$$

y se introduce un nuevo token *ab* en el vocabulario:

$$\mathcal{V}_{k+1} = \mathcal{V}_k \cup \{ab\}.$$

Este proceso se repite hasta alcanzar un tamaño de vocabulario predefinido $|\mathcal{V}|$.
Segmentación: Una vez aprendido el vocabulario, la tokenización de un texto consiste en aplicar de forma *greedy* (la mejor decisión) las fusiones aprendidas para obtener la secuencia de subpalabras.

Byte Pair Encoding (BPE)

Características

- Algoritmo determinista y eficiente.
- Produce una única segmentación por texto.
- Muy utilizado en modelos Transformer modernos.

Ejemplo: La palabra *imposible*. Inicialmente, el texto se representa como una secuencia de caracteres:

$$i\ m\ p\ o\ s\ i\ b\ l\ e.$$

Durante el entrenamiento, BPE aprende un conjunto ordenado de fusiones, entre ellas:

$$(impos, i) \to imposi, \quad (bl, e) \to ble.$$

Al aplicar estas fusiones de forma greedy durante la tokenización, se obtiene la segmentación:

$$\text{imposible} \longrightarrow [\text{imposi}]\ [\text{ble}].$$

Dado que el proceso es determinista, no se generan segmentaciones alternativas para una misma palabra.

WordPiece

Definición

El algoritmo *WordPiece* es un método de tokenización por subpalabras basado en la **optimización probabilística** de un vocabulario. Su objetivo es seleccionar subpalabras que maximicen la probabilidad del corpus de entrenamiento bajo un modelo de lenguaje.

Idea general: WordPiece busca un vocabulario $\mathcal{V}$ que maximice la verosimilitud del corpus:

$$\mathcal{L} = \sum_{x \in \mathcal{D}} \log P(x),$$

donde $\mathcal{D}$ es el conjunto de textos de entrenamiento.
A diferencia de BPE, la selección de subpalabras no se basa únicamente en frecuencia, sino en su impacto en la probabilidad del modelo.

WordPiece

Construcción del vocabulario: Se parte de un vocabulario inicial de caracteres. En cada iteración, se evalúan posibles fusiones de subpalabras y se selecciona aquella que produce la mayor mejora en la verosimilitud del corpus.
Las subpalabras que no aparecen al inicio de una palabra suelen marcarse con un prefijo especial, por ejemplo:

`##ción`, `##mente`.

Segmentación:
Durante la tokenización, WordPiece selecciona la segmentación que maximiza:

$$P(t_1, \ldots, t_T) = \prod_{i=1}^{T} P(t_i).$$

WordPiece

Características

- Algoritmo basado en criterios probabilísticos.
- Produce una única segmentación por texto.
- Utilizado en modelos como BERT.

Ejemplo: La palabra *jugando*.
Una posible segmentación es:

$$\text{jugando} \longrightarrow [\text{jug}]\ [\#\#\text{ando}].$$

El prefijo ## indica que la subpalabra no puede aparecer al inicio de una palabra.

Unigram Language Model

Definición

El algoritmo *Unigram Language Model* formula la tokenización como un **modelo probabilístico explícito**. Se asume que una secuencia de subpalabras se genera de forma independiente a partir de una distribución unigram.

Dada una secuencia $(t_1, \ldots, t_T)$, su probabilidad es:

$$P(t_1, \ldots, t_T) = \prod_{i=1}^{T} P(t_i).$$

El objetivo es maximizar la probabilidad del corpus completo bajo este modelo.

Unigram Language Model

Entrenamiento del vocabulario: Se parte de un vocabulario inicial grande $\mathcal{V}_{\text{init}}$ y se eliminan iterativamente los tokens menos útiles, optimizando:

$$\arg\max_{\mathcal{V}} \sum_{x \in \mathcal{D}} \log P(x).$$

Segmentación: A diferencia de BPE y WordPiece, Unigram permite **múltiples segmentaciones posibles**. Durante la tokenización se selecciona la más probable:

$$(t_1, \ldots, t_T) = \arg\max \prod_{i=1}^{T} P(t_i).$$

Unigram Language Model

Características

- Modelo probabilístico explícito.
- Permite segmentaciones alternativas.
- Utilizado en SentencePiece.

Ejemplo: La palabra *desconocido*.
Segmentaciones posibles:

$$[\text{des}][\text{conocido}], \quad [\text{desco}][\text{nocido}].$$

El modelo selecciona aquella con mayor probabilidad total.

Byte-level BPE

Definición

La tokenización a nivel de bytes es una variante de BPE en la que los símbolos iniciales no son caracteres, sino **bytes**. Esto garantiza cobertura completa de cualquier texto Unicode.

El vocabulario inicial se define como:

$$\mathcal{V}_0 = \{0, 1, \ldots, 255\},$$

correspondiente a todos los valores posibles de un byte.
Proceso: A partir del vocabulario inicial de bytes, se aplica el algoritmo BPE estándar, fusionando pares frecuentes de bytes para formar subpalabras más largas.
Ventaja principal: Cualquier texto puede tokenizarse sin necesidad de un token `<UNK>`, ya que todos los símbolos Unicode pueden representarse como secuencias de bytes.

Byte-level BPE

Características

- Cobertura total del espacio Unicode.
- Elimina completamente el problema de OOV.
- Tokens menos interpretables para humanos.
- Posible aumento de la longitud de secuencia.

Ejemplo: Un carácter no latino.

$$\alpha \longrightarrow [\text{byte}_1]\,[\text{byte}_2]\,[\text{byte}_3]$$

Estos bytes pueden posteriormente fusionarse en tokens más largos durante el entrenamiento.

Tokens especiales

Definición

Los *tokens especiales* son símbolos artificiales incluidos en el vocabulario del tokenizador que no representan contenido léxico, sino que cumplen funciones **estructurales** dentro del modelo de lenguaje.

Definición formal:
Sea $\mathcal{V}$ el vocabulario de tokens, se define el vocabulario extendido como:

$$\mathcal{V}' = \mathcal{V} \cup \{\texttt{<BOS>},\ \texttt{<EOS>},\ \texttt{<PAD>},\ \texttt{<UNK>}\}.$$

Estos tokens permiten controlar la estructura y el flujo de las secuencias procesadas por el modelo.

Tokens especiales

Principales tokens especiales y su función:

- <BOS> (*Beginning of Sequence*): marca el inicio de una secuencia.
- <EOS> (*End of Sequence*): indica el final de una secuencia.
- <PAD> (*Padding*): rellena secuencias para igualar longitudes en un batch.
- <UNK> (*Unknown*): representa tokens fuera del vocabulario (si existen).

Ejemplo de uso:

[<BOS>] Me gusta la pizza [<EOS>]

Capa de embeddings

Capa de embeddings

Motivación

Los modelos neuronales no pueden operar directamente sobre identificadores discretos. La *capa de embeddings* transforma cada token en una representación vectorial continua que permite al modelo aprender relaciones semánticas y sintácticas.

Idea clave:

$$\text{token (ID)} \longrightarrow \text{vector en } \mathbb{R}^d$$

Definición formal de embeddings

Definición

La capa de embeddings es una función que asigna a cada token del vocabulario un vector denso de dimensión fija.

Sea $\mathcal{V}$ un vocabulario de tamaño $|\mathcal{V}|$. La capa de embeddings se define como:

$$\mathcal{E} : \{1, \ldots, |\mathcal{V}|\} \to \mathbb{R}^d$$

Se implementa mediante una matriz entrenable:

$$E \in \mathbb{R}^{|\mathcal{V}| \times d}.$$

Lookup de embeddings

Obtención del embedding de un token:
Dado un token con identificador i, su embedding es:

$$e_i = E_{i,:}$$

Esta operación es un *lookup* (selección de fila), no una multiplicación matricial completa.
Secuencia de tokens:
Para una secuencia de IDs $(i_1, \ldots, i_T)$, la salida es:

$$X = \begin{bmatrix} E_{i_1,:} \\ E_{i_2,:} \\ \vdots \\ E_{i_T,:} \end{bmatrix} \in \mathbb{R}^{T \times d}.$$

Interpretación geométrica

Embeddings como espacio vectorial

Cada token se representa como un punto en un espacio vectorial de dimensión d.

La similitud entre tokens puede medirse mediante:

$$\langle e_i, e_j \rangle \quad \text{o} \quad \cos(e_i, e_j) = \frac{e_i \cdot e_j}{\|e_i\| \|e_j\|}$$

Intuición:

- Tokens con contextos similares $\rightarrow$ vectores cercanos.
- Relaciones lingüísticas $\rightarrow$ estructura geométrica.

Aprendizaje de los embeddings

Entrenamiento

Los embeddings se inicializan aleatoriamente y se optimizan junto con el resto del modelo mediante retropropagación.

Una función de pérdida típica en modelos de lenguaje es:

$$\mathcal{L} = -\sum_{t} \log P(i_t \mid i_{<t})$$

Actualización de parámetros:

$$E \leftarrow E - \eta \frac{\partial \mathcal{L}}{\partial E}$$

Importante: los embeddings no codifican significado explícito, sino patrones útiles para la tarea.

Embeddings posicionales

Problema del orden

El mecanismo de atención del Transformer es invariante al orden. Es necesario introducir información posicional.

Para cada posición t se define un vector:

$$p_t \in \mathbb{R}^d$$

La entrada final al modelo es:

$$x_t = e_{i_t} + p_t$$

Resumen: embeddings

Ideas clave

- Los embeddings convierten tokens discretos en vectores continuos.
- Se implementan como una matriz entrenable.
- Capturan relaciones semánticas y sintácticas.
- Se combinan con información posicional.

Pipeline completo:

$$\text{texto} \rightarrow \text{tokens} \rightarrow \text{IDs} \rightarrow \text{embeddings} \rightarrow \text{Transformer}$$

Las redes recurrentes para el procesamiento del lenguaje natural

¿Qué son las redes neuronales recurrentes?

Datos y arquitecturas neuronales (Repaso)

Tipos de datos

En aprendizaje profundo, la arquitectura de la red depende fuertemente de la estructura de los datos.

- **Datos tabulares**: vectores sin estructura temporal.
- **Datos espaciales**: imágenes, mapas (estructura 2D).
- **Datos secuenciales**: texto, audio, series temporales.

Cada tipo de dato requiere mecanismos inductivos distintos.

Redes feedforward (MLP) (Repaso)

Características

Las redes *feedforward* procesan entradas de tamaño fijo sin noción explícita de orden.

Formalmente:

$$y = f(Wx + b)$$

Limitaciones en PLN:

- Requieren entradas de longitud fija.
- No modelan dependencias entre posiciones.
- Pierden información de orden.

Redes Convolucionales (CNN) (Repaso)

Idea principal

Las CNN introducen un sesgo inductivo de **localidad** mediante filtros convolucionales compartidos.

En 1D (texto):

$$h_t = \sigma(W \cdot x_{t:t+k} + b)$$

Ventajas en PLN:

- Capturan patrones locales (n-gramas).
- Paralelizables.
- Buen rendimiento en tareas de clasificación.

Limitaciones de CNN en PLN

Problemas clave

- Ventana de contexto limitada (*receptive field*).
- Dificultad para capturar dependencias largas.
- El orden se modela de forma implícita y local.

Aunque pueden apilarse muchas capas, el contexto global no se modela de forma natural.

¿Por qué redes recurrentes?

Motivación

El lenguaje es una secuencia donde el significado de un elemento depende del contexto previo.

Las RNN introducen:

- un **estado interno** que evoluciona con la secuencia,
- una memoria del pasado.

Esto permite modelar dependencias a largo plazo de forma explícita.

Ejemplo intuitivo

Frase:

"El perro que persiguió al gato estaba cansado"

- Una CNN capta patrones locales como:
  - "perro que persiguió"
  - "al gato"
- Una RNN puede mantener información de:
  - el sujeto original (*perro*),
  - a lo largo de toda la secuencia.

Redes Neuronales Recurrentes (RNN)

Motivación

El lenguaje natural es secuencial: una palabra depende del contexto.

Ejemplo:

> *"No es lo mismo decir banco al hablar de dinero que de un parque."*

Las RNN están diseñadas para procesar secuencias manteniendo un **estado interno** que resume el pasado.

Aplicaciones en PLN:

- Modelos de lenguaje
- Etiquetado de secuencias
- Traducción automática
- Análisis de sentimiento

La RNN como sistema con estado

Vista conceptual

Una RNN puede verse como una **caja negra con memoria**.

En cada instante de tiempo t:

- recibe una entrada x_t
- actualiza su estado interno h_t
- opcionalmente produce una salida y_t

Idea clave

El estado h_t resume todo lo que la RNN ha visto hasta ahora.

Una sola célula recurrente

Definición simplificada

Una RNN es **una única red pequeña** que se reutiliza en cada paso temporal.

En el instante t, la célula recibe:

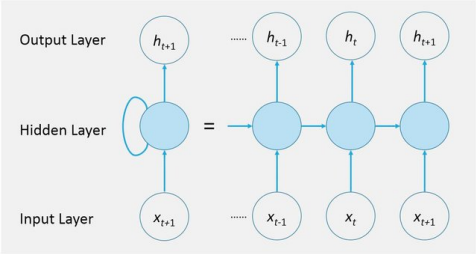
- x_t: la entrada actual
- h_{t-1}: la memoria previa

Produce:

- h_t: nueva memoria actualizada

Importante

No hay muchas redes: es siempre la misma.



RNN desenrollada en el tiempo

Definición matemática de una RNN

La actualización del estado se define como:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

- W_x: cómo entra la información nueva
- W_h: cómo se usa la memoria
- tanh: mantiene el estado estable

Importante

Los mismos parámetros se usan en todos los pasos temporales.

RNN desenrollada en el tiempo

Visualización

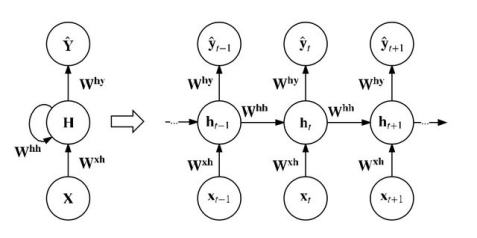
La RNN puede verse como la misma célula aplicada repetidamente:

$$(x_1, h_0) \to h_1 \to h_2 \to \cdots \to h_T$$

- No son redes distintas
- Son copias conceptuales de la misma célula

Importante

La profundidad de una RNN es temporal, no espacial.



RNN desenrollada en el tiempo

Tipos de tareas con RNN

One-to-Many (generación):

$$x \to (y_1, \ldots, y_T)$$

Ejemplo: generación de texto, image captioning

Many-to-One (clasificación):

$$(x_1, \ldots, x_T) \to y$$

Ejemplo: análisis de sentimiento

Many-to-Many (etiquetado):

$$(x_1, \ldots, x_T) \to (y_1, \ldots, y_T)$$

Ejemplo: POS tagging, NER

Entrenamiento de RNN: Backpropagation Through Time (BPTT)

Idea clave

BPTT es el algoritmo usado para entrenar RNN.

- Es backpropagation estándar
- Aplicado a una RNN desenrollada en el tiempo

Importante

No es un algoritmo nuevo: es backpropagation a lo largo del tiempo.

Entrenamiento: Backpropagation Through Time (BPTT)

Paso 1: Cálculo de la salida y la pérdida

En muchas tareas, la RNN produce la salida al final de la secuencia.

$$y = g(h_T) \qquad \mathcal{L}(y, y^*)$$

- El error se calcula en el último paso
- La pérdida mide qué tan buena fue la predicción

Pregunta

¿Cómo afecta este error a los pasos anteriores?

Entrenamiento: Backpropagation Through Time (BPTT)

Paso 2: Propagación del error hacia atrás

El error se propaga desde el final hacia el inicio de la secuencia.

$$h_T \to h_{T-1} \to \cdots \to h_1$$

- El gradiente fluye hacia atrás en el tiempo
- Los mismos pesos se ajustan en todos los pasos

Idea clave

El pasado influye en el error del presente.

Entrenamiento: Backpropagation Through Time (BPTT)

Paso 3: Multiplicación repetida de gradientes

Cada paso hacia atrás multiplica el gradiente por los pesos recurrentes.

$$\frac{\partial h_t}{\partial h_{t-k}} \approx (W_h)^k$$

- Retroceder 1 paso: $\times W_h$
- Retroceder muchos pasos: $\times (W_h)^k$
- Simplificación: para ir k pasos atrás en el tiempo, el gradiente se multiplica k veces por los mismos pesos

Intuición

El gradiente se transforma paso a paso.

Entrenamiento: Backpropagation Through Time (BPTT)

Paso 4: Gradiente que desaparece (Vanishing gradient)

Si los pesos recurrentes son pequeños:

$$\|W_h\| < 1$$

- El gradiente se hace cada vez más pequeño
- La información lejana se pierde

Consecuencia

La RNN olvida el pasado lejano.

Entrenamiento: Backpropagation Through Time (BPTT)

Paso 5: Gradiente que explota

Si los pesos recurrentes son grandes:

$$\|W_h\| > 1$$

- El gradiente crece rápidamente
- El entrenamiento se vuelve inestable

Consecuencia

Los pesos cambian de forma descontrolada.

Entrenamiento de una RNN en PyTorch

Idea clave

En PyTorch, **BPTT ocurre automáticamente** al llamar a `loss.backward()`.

Entrenamiento típico:

```
h = h0
for x_t in sequence:
    h = rnn_cell(x_t, h)

loss = criterion(h, y)
loss.backward()
optimizer.step()
```

- Cada iteración añade nodos al grafo computacional
- El estado oculto conecta los pasos temporales
- El backward recorre el grafo hacia atrás en el tiempo

Este `backward()` es BTT.

Problema en PyTorch: Vanishing Gradient

Qué ocurre

Durante `loss.backward()`, los gradientes se hacen muy pequeños.

Síntomas en PyTorch:

```
for name, p in model.named_parameters():
    print(name, p.grad.norm())
```

- Gradientes cercanos a 0
- Pesos apenas se actualizan
- El modelo no aprende dependencias largas

Causa

Multiplicaciones repetidas de valores menores que 1 en BPTT.

Problema en PyTorch: Exploding Gradient

Qué ocurre

Durante `loss.backward()`, los gradientes crecen sin control.

Síntomas en PyTorch:

```
loss = NaN
RuntimeError: overflow detected
```

- Gradientes enormes
- Pesos inestables
- Entrenamiento diverge

Solución típica:

```
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
```

Idea clave

El gradiente explota al multiplicarse muchas veces por valores > 1.

Limitaciones de las RNN

- Dificultad para capturar dependencias largas
- Gradientes inestables
- Memoria limitada

Consecuencia

Se introducen arquitecturas más avanzadas.

¿Cómo superar estas limitaciones?

Pregunta clave

¿Cómo diseñar un modelo que pueda decidir qué recordar y qué olvidar?

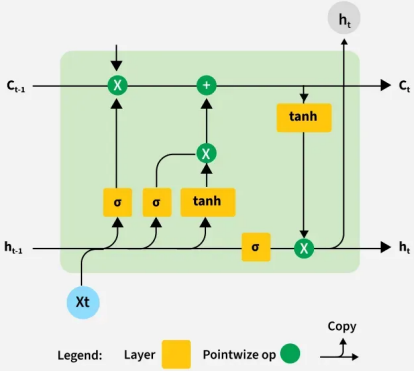
- Control explícito de la memoria
- Flujo de gradiente más estable

LSTM

Idea principal

Una LSTM separa la **memoria a largo plazo** (c_t) del **estado oculto** (h_t).

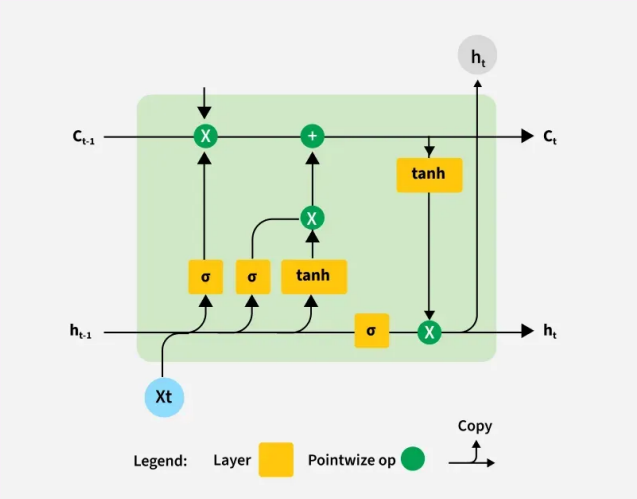
- c_t: "cinta transportadora" de memoria (más estable)
- Compuertas controlan el flujo de información:
  - **Forget gate**: qué olvidar
  - **Input gate**: qué escribir
  - **Output gate**: qué mostrar como salida



Importante

La LSTM aprende a decidir **qué recordar** y **qué olvidar**.

LSTM


h_t
C_{t-1}
C_t
tanh
σ
σ
tanh
σ
h_{t-1}
h_t
Xt
Copy
Legend:
Layer
Pointwize op

LSTM: Fórmulas

$$f_t = \sigma(W_f[x_t, h_{t-1}] + b_f) \qquad i_t = \sigma(W_i[x_t, h_{t-1}] + b_i)$$

$$\tilde{c}_t = \tanh(W_c[x_t, h_{t-1}] + b_c)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$o_t = \sigma(W_o[x_t, h_{t-1}] + b_o) \qquad h_t = o_t \odot \tanh(c_t)$$

Intuición

c_t tiene un camino más "directo" en el tiempo, lo que ayuda a evitar el vanishing gradient.

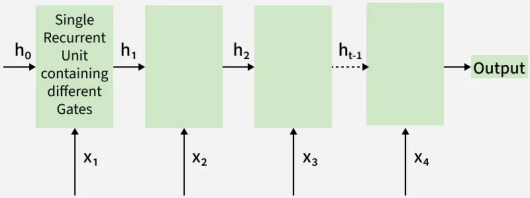
GRU

Idea principal

La GRU simplifica la LSTM: combina memoria y estado en una sola variable (h_t).

Compuertas:

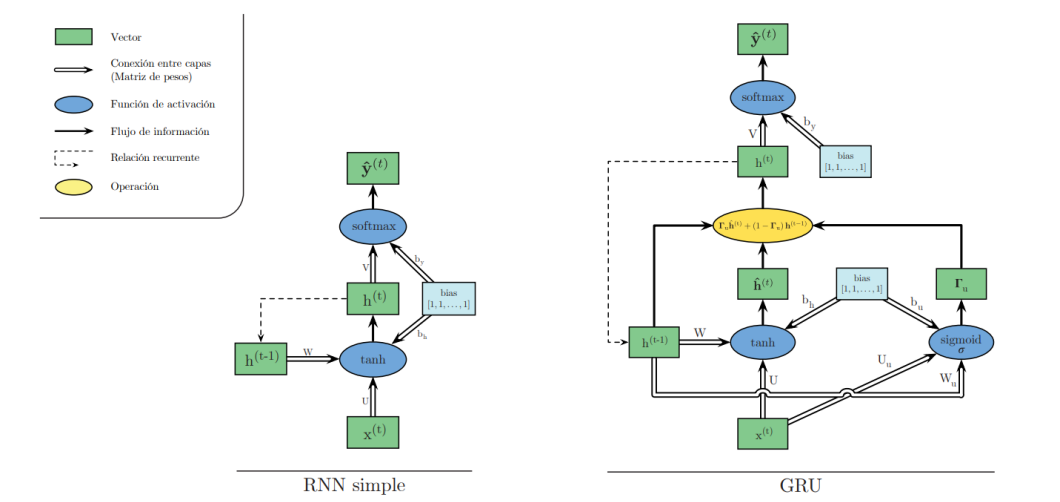
- **Update gate** (z_t): cuánto conservar del pasado
- **Reset gate** (r_t): cuánto olvidar al calcular la nueva información



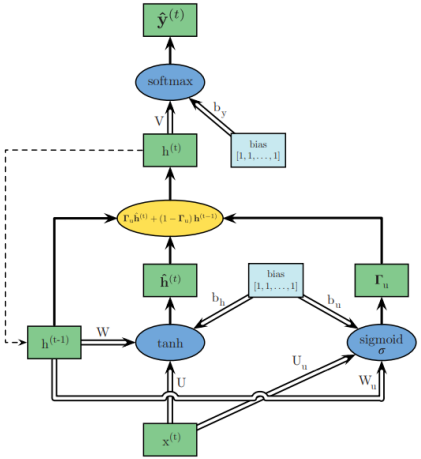
Importante

GRU suele ser más simple y rápida, con rendimiento similar en muchas tareas.

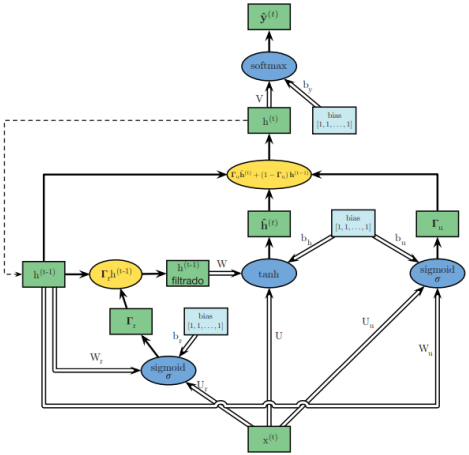
GRU


Vector
Conexión entre capas (Matriz de pesos)
Función de activación
Flujo de información
Relación recurrente
Operación
$\hat{y}^{(t)}$
softmax
V
b_y
$h^{(t)}$
bias $[1, 1, \ldots, 1]$
b_h
$h^{(t-1)}$
W
tanh
U
$x^{(t)}$
RNN simple
$\hat{y}^{(t)}$
softmax
V
b_y
$h^{(t)}$
bias $[1, 1, \ldots, 1]$
$\Gamma_u h^{(t)} + (1 - \Gamma_u) h^{(t-1)}$
$\hat{h}^{(t)}$
bias $[1, 1, \ldots, 1]$
$b_{\hat{h}}$
b_u
Γ_u
$h^{(t-1)}$
W
tanh
sigmoid σ
U_u
U
W_u
$x^{(t)}$
GRU

GRU

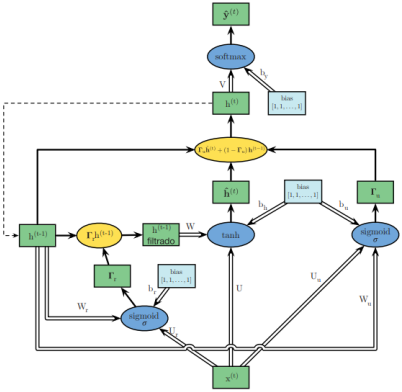


GRU



GRU con puerta de relevancia

GRU: Fórmulas



GRU con puerta de relevancia

$$\mathbf{\Gamma}_{\mathrm{u}} = \sigma(\mathbf{M}_u[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}] + \mathbf{b}_u)$$

$$\mathbf{\Gamma}_{\mathrm{r}} = \sigma(\mathbf{M}_r[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}] + \mathbf{b}_r)$$

$$\hat{\boldsymbol{h}}^{(t)} = \tanh(\mathbf{M}[\mathbf{\Gamma}_{\mathrm{r}} \odot \mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}] + \mathbf{b}_h)$$

$$\boldsymbol{h}^{(t)} = \mathbf{\Gamma}_{\mathrm{r}} \odot \hat{\mathbf{h}}^{(t)} + (1 - \mathbf{\Gamma}_{\mathrm{u}}) \odot \mathbf{h}^{(t-1)}$$

donde:

$$\mathbf{M} = [\mathbf{W}, \mathbf{U}]$$

$$\mathbf{M}_r = [\mathbf{W}_r, \mathbf{U}_r]$$

$$\mathbf{M}_u = [\mathbf{W}_u, \mathbf{U}_u]$$

Intuición

z_t actúa como un "mezclador" entre lo viejo (h_{t-1}) y lo nuevo $(\tilde{h}_t)$.

LSTM vs GRU

Característica	LSTM	GRU
Puertas	3 (Entrada, Olvido, Salida)	2 (Actualización, Reinicio)
Estado de Celda	Sí, tiene estado de celda	No (solo estado oculto)
Velocidad de Entrenamiento	Más lenta debido a su complejidad	Más rápida debido a arquitectura más simple
Carga Computacional	Mayor por más puertas y parámetros	Menor por menos puertas y parámetros
Rendimiento	A menudo mejor en tareas que requieren memoria a largo plazo	Similar en muchas tareas con menor complejidad

PyTorch: LSTM para clasificación (Many-to-One)

Entrada: secuencia $(x_1, \ldots, x_T)$ **Salida: etiqueta** y

```
import torch
import torch.nn as nn
class LSTMClassifier(nn.Module):
    def __init__(self, input_dim, hidden_dim, num_classes):
        super().__init__()
        self.lstm = nn.LSTM(input_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, num_classes)
    def forward(self, x):
        # x: (batch, T, input_dim)
        out, (h_T, c_T) = self.lstm(x)
        # h_T: (1, batch, hidden_dim)
        logits = self.fc(h_T[-1])
        return logits
model = LSTMClassifier(input_dim=300, hidden_dim=128, num_classes=2)
```

Usamos el último estado oculto h_T **como resumen de la secuencia.**

PyTorch: GRU para clasificación (Many-to-One)

```python
import torch
import torch.nn as nn

class GRUClassifier(nn.Module):
    def __init__(self, input_dim, hidden_dim, num_classes):
        super().__init__()
        self.gru = nn.GRU(input_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, num_classes)
    def forward(self, x):
        out, h_T = self.gru(x)
        logits = self.fc(h_T[-1])
        return logits
model = GRUClassifier(input_dim=300, hidden_dim=128, num_classes=2)
```

GRU no tiene estado de celda c_t; solo devuelve h_t.

PyTorch: entrenamiento (LSTM/GRU)

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for x, y in dataloader:
    optimizer.zero_grad()

    logits = model(x)          # forward
    loss = criterion(logits, y)

    loss.backward()            # BPTT via autograd
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)  # opcional

    optimizer.step()
```

`loss.backward()` **hace BPTT automáticamente sobre la secuencia.**

Nota práctica: secuencias largas

- El grafo crece con T (longitud de la secuencia)
- Más memoria GPU/CPU y más riesgo de vanishing/exploding

Soluciones prácticas

- **Gradient clipping** (para exploding)
- **Truncated BPTT** (procesar en trozos) + `detach()`

Modelos del lenguaje autorregresivos

¿Qué es un modelo de lenguaje?

Modelos del Lenguaje

Definición

Un **modelo de lenguaje** asigna probabilidades a secuencias de palabras.

$$P(w_1, w_2, \ldots, w_T)$$

Ejemplo:

"Hoy hace mucho ____"

- frío
- calor
- ruido

Idea clave

El modelo decide qué palabra es más probable.

Modelos Autorregresivos

Idea principal

Un modelo autorregresivo predice la siguiente palabra usando solo las palabras anteriores.

$$P(w_1, \ldots, w_T) = \prod_{t=1}^{T} P(w_t \mid w_1, \ldots, w_{t-1})$$

Importante

El pasado explica el futuro, palabra a palabra.

Ejemplo: predicción palabra a palabra

Frase:

"El gato duerme en el ____"

El modelo calcula:

$$P(\text{El})$$

$$P(\text{gato} \mid \text{El})$$

$$P(\text{duerme} \mid \text{El gato})$$

$$P(\text{sofá} \mid \text{El gato duerme en el})$$

Idea clave

Cada palabra se predice usando todo el contexto previo.

¿Por qué autorregresivo?

Generación

El modelo usa sus propias predicciones como entrada.

- Predice w_1
- Usa w_1 para predecir w_2
- Usa (w_1, w_2) para predecir w_3

Clave

El modelo se alimenta de sí mismo.

Modelos autorregresivos con RNN/LSTM

Cómo se implementa

Usamos una RNN (o LSTM/GRU) para resumir el pasado.

$$h_t = f(h_{t-1}, w_{t-1})$$

$$P(w_t \mid w_{<t}) = \text{softmax}(Wh_t)$$

Idea clave

El estado oculto h_t representa el contexto.

Entrenamiento de un modelo autorregresivo

Teacher Forcing

Durante el entrenamiento, se da al modelo la palabra correcta anterior.

Pérdida:

$$\mathcal{L} = -\sum_{t=1}^{T} \log P(w_t \mid w_{<t})$$

- Cross-entropy
- Se entrena a predecir la siguiente palabra

Clave

Aprender a predecir la siguiente palabra es aprender el lenguaje.

Generación de texto

Cómo se genera

Una vez entrenado, el modelo genera texto palabra a palabra.

- Se empieza con un token inicial
- Se predice la siguiente palabra
- Se muestrea (argmax, sampling)
- Se repite

Importante

Generar texto es repetir la predicción de la siguiente palabra.

Limitaciones

- Dificultad con dependencias largas
- Generación secuencial (lenta)
- Errores se acumulan

Consecuencia

Se desarrollan modelos más potentes: Transformers.

Modelo de Lenguaje Autorregresivo con RNN (PyTorch)

Objetivo

Aprender:

$$P(w_t \mid w_{<t}) \quad \text{(predecir el siguiente token)}$$

- Entrada: secuencia de tokens $w_1, \ldots, w_{T-1}$
- Salida: logits para predecir $w_2, \ldots, w_T$

Idea clave

Entrenamos a la RNN para que en cada paso prediga el siguiente token.

PyTorch: LSTM Language Model

```python
import torch
import torch.nn as nn
class LSTMLanguageModel(nn.Module):
    def __init__(self, vocab_size, emb_dim=128, hidden_dim=256,
                 num_layers=1, dropout=0.0):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, emb_dim)
        self.lstm = nn.LSTM(
            input_size=emb_dim,
            hidden_size=hidden_dim,
            num_layers=num_layers,
            batch_first=True,
            dropout=dropout if num_layers > 1 else 0.0
        )
        self.fc = nn.Linear(hidden_dim, vocab_size)
    def forward(self, x, state=None):
        # x: (batch, T) tokens
        x_emb = self.embed(x)                  # (batch, T, emb_dim)
        out, state = self.lstm(x_emb, state)   # out: (batch, T, hidden)
        logits = self.fc(out)                  # (batch, T, vocab_size)
        return logits, state
```

Salida: `logits[:, t, :]` **son las puntuaciones para predecir** w_{t+1}**.**

PyTorch: entrenamiento autorregresivo (LSTM)

Para cada secuencia `[w1, w2, ..., wT]`: input=`[w1, ..., w(T-1)]`, target=`[w2, ..., wT]`

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

model.train()
for x_full in dataloader:                # x_full: (batch, T)
    x_in = x_full[:, :-1]                # (batch, T-1)
    y_tg = x_full[:,  1:]                # (batch, T-1)

    optimizer.zero_grad()
    logits, _ = model(x_in)              # (batch, T-1, vocab)

    loss = criterion(
        logits.reshape(-1, logits.size(-1)),  # (batch*(T-1), vocab)
        y_tg.reshape(-1)                      # (batch*(T-1))
    )

    loss.backward()                       # BPTT via autograd
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)  # opcional
    optimizer.step()
```

`loss.backward()` realiza BPTT sobre todos los pasos temporales.

PyTorch: generación autoregresiva (LSTM)

```python
@torch.no_grad()
def generate(model, start_tokens, max_new_tokens=30, temperature=1.0):
    model.eval()

    x = torch.tensor([start_tokens])   # (1, T)
    state = None

    for _ in range(max_new_tokens):
        # Pasamos SOLO el ultimo token, pero mantenemos "state"
        logits, state = model(x[:, -1:], state)     # (1, 1, vocab)
        next_logits = logits[:, -1, :] / temperature
        probs = torch.softmax(next_logits, dim=-1)
        next_token = torch.multinomial(probs, num_samples=1)  # (1,1)

        x = torch.cat([x, next_token], dim=1)

    return x.squeeze(0).tolist()
```

Idea clave

La LSTM mantiene el contexto en `state=(h,c)` mientras generamos token a token.

Evaluación: Perplejidad

Definición

La perplejidad mide qué tan bien el modelo predice los tokens.

$$\text{PPL} = \exp\left(\frac{1}{N}\sum_{t=1}^{N} -\log P(w_t \mid w_{<t})\right)$$

- Menor PPL $\Rightarrow$ mejor modelo
- Es el estándar en modelos de lenguaje